
CASE STUDY / 2026

NTA

Nova Tech Agency

Full-stack agency platform — content-driven, animation-rich, continuously deployed.

React 18 • Spring Boot 3 • PostgreSQL • Docker CI/CD — Full-stack web platform

PREPARED BY

Mudasir Shafiq

Full-stack & data engineer · chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	System Architecture	Page 4
03	CI/CD Pipeline	Page 5
04	Data Model & API Endpoints	Page 6
05	Technology Stack	Page 7
06	The Challenge	Page 8
07	The Approach	Page 9
08	Code Spotlight	Page 10
09	Outcome & Engagement	Page 11

This document presents an in-depth look at the Nova Tech Agency platform — its architecture, data model, API surface, deployment pipeline, and the engineering decisions behind it. Every diagram, code snippet, and technical claim is grounded directly in the project source.

SECTION 01

Project Overview

What NTA is, who it's for, and the shape of the platform.

Nova Tech Agency (NTA) is a public-facing marketing and lead-generation platform for a digital agency. Rather than ship a static brochure, the front end is a polished React 18 single-page app that live-reads every section — services, portfolio, about, contact — from a Spring Boot REST API backed by managed PostgreSQL. The business can update its services list, case studies, statistics, and contact copy at any time without a single deploy.

The entire stack is containerised. A single GitHub Actions workflow builds both frontend and backend Docker images on every push to **main**, pushes them to Docker Hub, and atomically swaps the running containers on the host. The path from a Git commit to a live update is one merge.

Aesthetically the site leans into a custom design-studio feel — a Three.js starfield hero, Framer Motion scroll reveals, a navy/accent design system tuned in Tailwind — while operationally behaving like a SaaS product: typed endpoints, JPA-managed schema, multi-stage Docker builds, and CORS-aware integration.

11

REST endpoints

7

JPA entities

CI/CD

On every push

Neon

Managed Postgres

SECTION 02

System Architecture

Two containers, one database, edge-served SPA + JSON API.

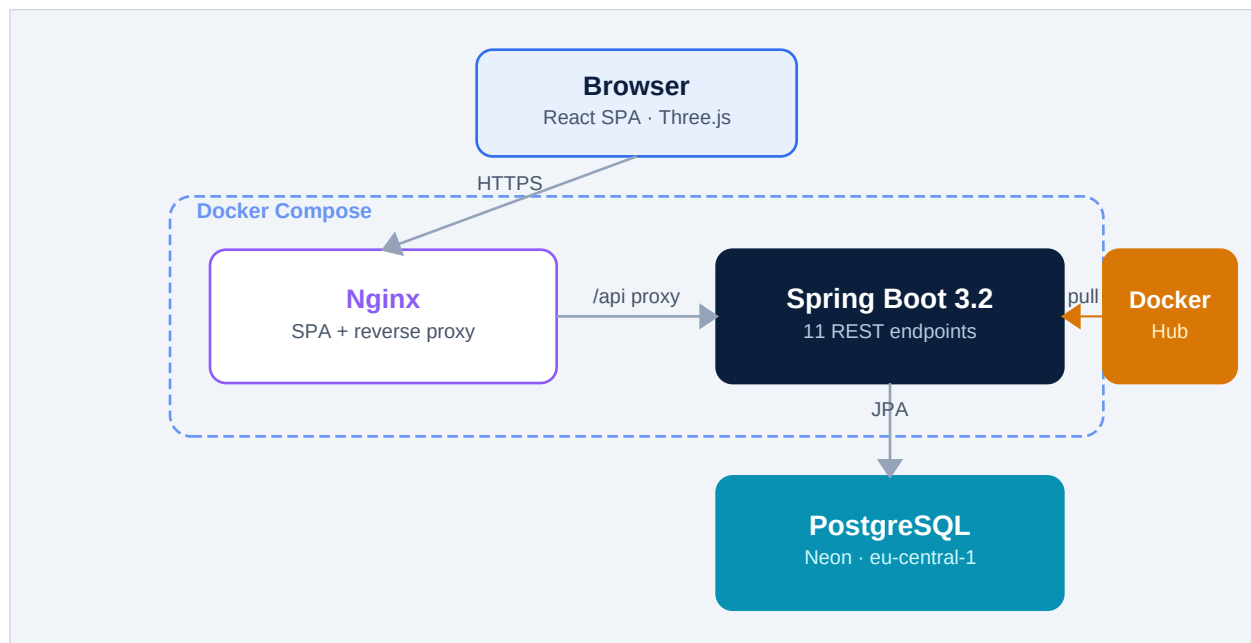


Figure — Browser → Nginx (SPA + reverse proxy) → Spring Boot → PostgreSQL on Neon; both containers built and pulled from Docker Hub.

The architecture is deliberately straightforward. The browser loads the React SPA from **Nginx**, which also reverse-proxies `/api/*` requests to the Spring Boot service. Spring Boot exposes eleven typed REST endpoints, mapped through Spring Data JPA to a **PostgreSQL** instance hosted on Neon (eu-central-1). Both containers are produced by multi-stage Dockerfiles (Maven → JRE-Alpine for the backend; Node → Nginx for the frontend) and run via Docker Compose locally; in production they are pulled from Docker Hub.

SECTION 03

CI/CD Pipeline

Every commit to main rebuilds both images, pushes them, and atomically swaps the live containers.

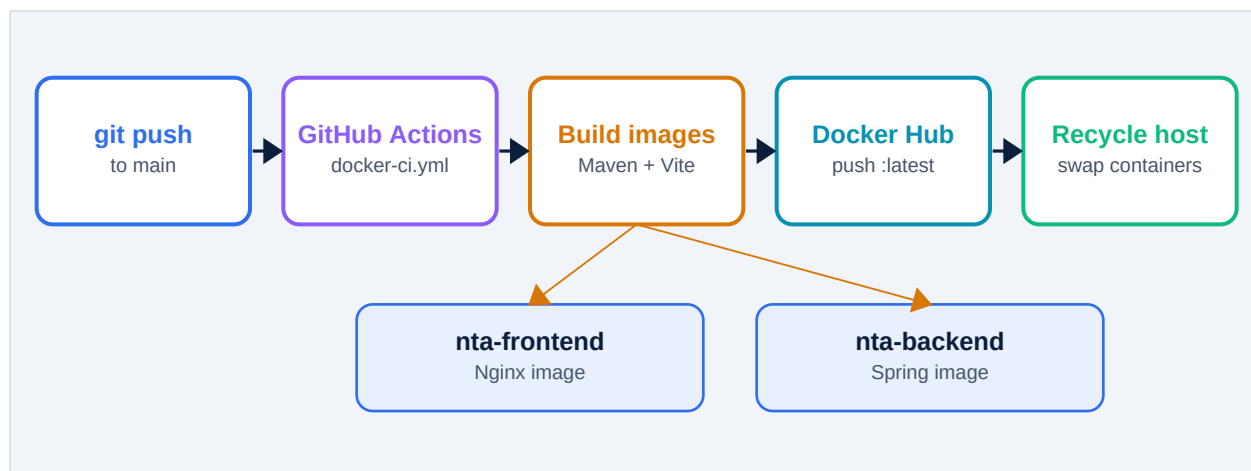


Figure — GitHub Actions docker-ci.yml — git push triggers parallel image builds, Docker Hub push, and host-side container swap.

The deployment story is the most distinctive part of NTA. A single workflow (`.github/workflows/docker-ci.yml`) wires the entire path: checkout, set up QEMU and Buildx for cross-arch, log in to Docker Hub, build and push **chmudasir007/nta-frontend** and **chmudasir007/nta-backend**, then on the host run a sequence of `docker stop / rm / run` commands to atomically swap each container. The same workflow runs on pull requests so every change is validated before it lands.

Operationally this means content updates that go through the database are zero-deploy, and code updates are one-merge-to-deploy. No manual server work is involved in either path.

SECTION 04

Data Model & API Endpoints

Seven JPA entities map cleanly to eleven REST endpoints across five resources.

JPA entities

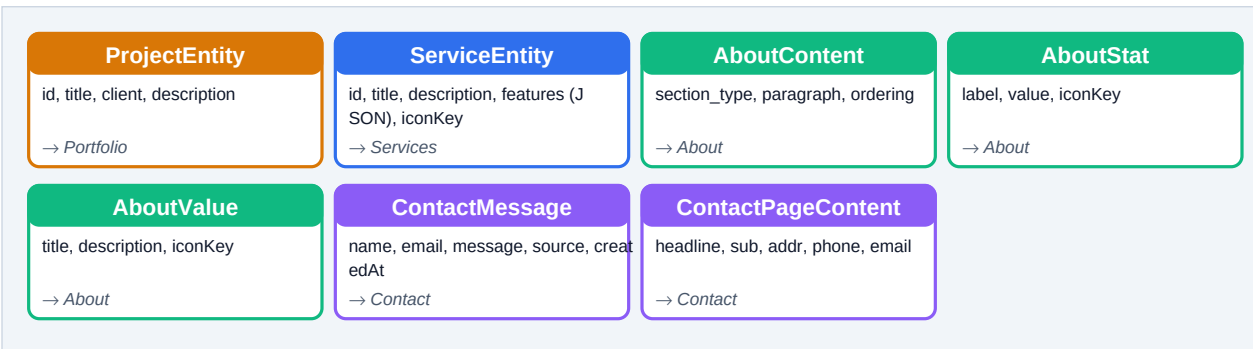


Figure — Seven entities grouped by the five public resources they serve.

REST endpoints

Method	Endpoint	Returns
GET	/api/portfolio	ProjectDTO[] — all case studies
GET	/api/services	ServiceDTO[] — six services with parsed features
GET	/api/services/test	Diagnostic copy of /api/services
GET	/api/about	AboutResponse — stats + values + content
GET	/api/about/stats	AboutStat[]
GET	/api/about/values	AboutValue[]
GET	/api/about/content	AboutContent[] — story + mission
GET	/api/contact/content	ContactPageContent
POST	/api/contact/submit	ContactMessage — persists user message
GET	/api/contact/messages	ContactMessage[] — admin view
GET	/api/status	{status: 'Backend OK'} — healthcheck

SECTION 05

Technology Stack

Modern, container-ready, typed end-to-end.

Layer	Technology & Details
Frontend	React 18, Vite 5, Tailwind 3, Framer Motion 11, Three.js (starfield hero), Lucide, Axios
Backend	Java 17, Spring Boot 3.2, Spring Data JPA, Hibernate, Jackson
Database	PostgreSQL on Neon (eu-central-1) — 7 JPA entities, schema auto-managed via <code>ddl-auto=update</code>
API Surface	11 REST endpoints across <code>/api/portfolio</code> , <code>/api/services</code> , <code>/api/about</code> , <code>/api/contact</code> , <code>/api/status</code>
DevOps	Docker multi-stage builds (Maven→JRE-Alpine; Node→Nginx), docker-compose, GitHub Actions
Edge	Nginx in front of the SPA, reverse-proxying <code>/api/*</code> to the Spring service on <code>:8080</code>
Registry & Deploy	Docker Hub — <code>chmudasir007/nta-frontend</code> and <code>chmudasir007/nta-backend</code> ; auto-redeployed on push to main
Live API	<code>api.novatechagency.com</code> — CORS open via <code>@CrossOrigin(origins = "*")</code>

Why this stack. React + Vite produces a fast, animation-rich SPA without the build complexity of a full meta-framework. Spring Boot 3 on Java 17 gives a typed, well-tested API with built-in JPA and zero-config dependency injection. Neon-hosted Postgres removes any DBA overhead. Docker Compose plus GitHub Actions gives a one-merge deployment without paid CI/CD tooling.

SECTION 06

The Challenge

Agencies refresh their site constantly; engineers shouldn't sit on the critical path.

Agency websites are the most-edited surface a tech business owns. Services get added or dropped, stats roll forward, case studies are reshaped, calls-to-action are tested, and the team page shuffles every quarter. A hand-coded static site forces a developer into the loop for every wording change, which is both expensive and slow.

The brief was a site that is fast, animation-rich, and looks bespoke — but stays editable by non-developers. Editorial copy needs to live in a database, not in JSX files; engineering changes need to ship through a one-click pipeline, not an SSH session. The site also has to look the part: an agency selling premium tech work cannot ship a site that looks like a template.

Three constraints made the design non-trivial: **(1)** the SPA needs to load fast despite Three.js and Framer Motion on the hero; **(2)** every content fetch has to survive the first paint without a flash of empty state; **(3)** deploys must be atomic — no half-updated containers serving stale assets next to fresh API responses.

SECTION 07

The Approach

Clean separation of concerns, typed contracts, and a deployment pipeline that does the rest.

Database-driven content

Every public section — Hero copy, Services grid, Portfolio cards, About story / mission / stats / values, Contact page text and form submissions — reads from a JPA-backed REST endpoint. Seven entities, eleven endpoints, one query each. The frontend uses fetch / axios to load and render; the backend handles parsing JSON features-arrays, CORS, and persistence.

Production-grade backend

Spring Boot 3.2 on Java 17 with Spring Data JPA repositories, DTO mapping in the controllers, and a clean services layer for the contact-message lifecycle. Postgres driver pinned at 42.7.3, H2 retained for fast local development. A CockroachDB-compatible TLS certificate is baked into the image so the same container can later move to a Cockroach cluster without code changes.

Animation without bloat

Tailwind drives the design system; Framer Motion handles every scroll-aware reveal; Three.js powers the starfield hero. Each section is a stateful React component that lazy-loads its content over fetch — the hero paints immediately while the rest of the page hydrates.

True CI/CD

Multi-stage Docker builds keep both images small: Maven build → JRE-Alpine runtime for the backend; Node build → Nginx for the frontend. The GitHub Actions workflow recycles containers atomically on the host (*docker stop / rm / run*), so users never see a half-updated state.

SECTION 08

Code Spotlight

How the portfolio endpoint maps entities to DTOs, and how the workflow ships every commit.

Spring Boot portfolio endpoint — entity → DTO mapping

com/nta/controller/PortfolioController.java

```
@RestController
@RequestMapping("/api/portfolio")
@CrossOrigin(origins = "**")
public class PortfolioController {
    @Autowired
    private ProjectRepository projectRepository;

    @GetMapping
    public List<ProjectDTO> getAllProjects() {
        return projectRepository.findAll().stream()
            .map(p -> new ProjectDTO(p.getId(), p.getTitle(),
                p.getClient(), p.getDescription()))
            .collect(Collectors.toList());
    }
}
```

GitHub Actions workflow — build, push, swap (excerpt)

.github/workflows/docker-ci.yml

```
# Build & deploy frontend and backend on every push to main
on: { push: { branches: [main] } }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: docker/login-action@v2
      - uses: docker/build-push-action@v5
      with:
        context: ./frontend
        tags: chmudasir007/nta-frontend:latest
      - run: docker stop nta-frontend-1 || true
      - run: docker rm nta-frontend-1 || true
      - run: docker run -d --name nta-frontend-1 -p 5173:80 chmudasir007/nta-frontend:latest
```

SECTION 09

Outcome & Engagement

A site that looks bespoke and operates like a SaaS product.

NTA delivers an agency platform where editorial content moves at the speed of a database update, and engineering changes ship through a one-click pipeline. The frontend is fast, animation-rich, and looks custom; the backend is typed, JPA-managed, and observable; the deployment story is one merge to *main*.

Structurally, the codebase is ready for a CMS-style admin panel without rework. Every entity already has a REST endpoint; every endpoint already returns a DTO; the contact-message store already has an admin GET. The next iteration — a */admin* route that POSTs back to those endpoints — is additive, not destructive.

- › **Database-driven content** on every public section — copy updates are zero-deploy.
- › **Eleven typed REST endpoints** with DTO mapping and CORS-aware integration.
- › **Multi-stage Docker builds** keep images small and reproducible across machines.
- › **One-merge deployment** — GitHub Actions builds, pushes, and atomically swaps containers.
- › **Production-grade backend** — Spring Boot 3.2 on Java 17 with Spring Data JPA and Hibernate.

LET'S TALK

Mudasir Shafiq

Full-stack & data engineer

Email chmudasir.fr@gmail.com

Stacks React • Spring Boot • Python • Airflow • BigQuery • Docker